

Analyzing TDHIA IDAT files

Vignette Author

2026-05-27

Contents

Dataset generation	1
ICR-level regression tests	3

Here we will demonstrate how to process IDAT files from the True Diagnostic Human Imprintome Array (TDHIA) with the `tdhia` package.

The code is not evaluated during package checks because several steps require annotation resources and Sesame/Bioconductor cache files that may not be available on offline build systems. Copy the chunks into an interactive R session after installing the package and required annotation data.

Dataset generation

We begin by first generating simulated data. Sesame imports IDAT files into a SigSet data object. We are going to artificially generate SigSet files. Let's do a dataset of 10 SigSets (IDATS).

```
n_patients = 20
ss <- tdhia::simulate_idat_data(n_patients)
```

We can now load this data into `tdhia` and calculate the beta values and signal detection p-values. This is returned in a named list, with the following fields:

probe_beta_df: dataframe of beta values, `probe_id` x `sample_id`.
probe_pval_df: dataframe of signal p-values, `probe_id` x `sample_id`.
platform: string that describes platform for methylation array.
manifest: dataframe of the manifest file used for SeSesame processing.

```
probe_beta <-
  tdhia::load_idata_to_probes(idat_dir_paths = ss, multicore = 2,
                             quantile_norm = FALSE, db_flag = TRUE)
```

You can access the probe beta matrix with the **probe_beta_df** field.

```
# Access probe beta matrix
head(probe_beta$probe_beta_df)
```

We can filter out low quality probes with the `filt_probe_beta()`. This function returns a named list with the same format as `probe_beta` above.

```
# 2) Filter probes that are not mapped and discard poor signal
# Output: rows: probe_id      x      columns: patient
filt_probe_beta <-
  tdhia::filter_probes(probe_beta = probe_beta, discard_unmapped_probes = TRUE ,
                       max_sig_pval = 0.2, set_failed_betas_na = FALSE,
                       max_probe_fail_rate = 0.25,
                       discard_failed_probes = TRUE, db_flag = FALSE)
```

You can access the filtered probe beta matrix with the **probe_beta_df** field.

```
# Access filtered probe beta matrix
head(filt_probe_beta$probe_beta_df)
```

Once probe data has been filtered convert the probe beta matrix to a cpg data matrix, stored in a named list of the form:

cpg_beta_df: dataframe of beta values, probe_id x sample_id.
platform: string that describes platform for methylation array.
manifest: dataframe of the manifest file used for SeSama processing.
cpg_n_probes: number of probes for each cpg site.

Probe beta values that belong to the same cpg site for a given patient are averaged.

```
#3 Convert probe beta matrix to a cpg beta matrix
# Output: rows: cpg_id      x      columns: patient
cpg_beta <- tdhia::convert_probes_to_cpgs(filt_probe_beta, quantile_norm = FALSE,
                                         db_flag = TRUE, smooth_adj_cpgs = FALSE, multicore = 2)
```

You can access the filtered probe beta matrix with the **cpg_beta_df** field.

```
# Access filtered probe beta matrix
head(cpg_beta$cpg_beta_df)
```

Finally, we can convert the cpg beta matrix to an icr beta matrix, which is returned in a named list of the form:

icr_beta_df: dataframe of beta values, icr_id x sample_id.
platform: string that describes platform for methylation array.
manifest: dataframe of the manifest file used for SeSama processing.
n_CpGs: number of cpg sites in each icr site.

Probe beta values that belong to the same cpg site for a given patient are averaged.

```
#4 Convert probe beta matrix to an icr beta matrix
# Output: rows: icr_id      x      columns: patient
icr_beta <- tdhia::convert_cpgs_to_icrs(cpg_beta, max_icr_fail_rate = 0.2)
```

You can access the filtered probe beta matrix with the **probe_beta_df** field.

```
# Access filtered probe beta matrix
head(icr_beta$icr_beta_df)
```

Let's simulate a case control analysis, where we will test for differences in CpG methylation state between a control group and an treatment group.

```
groud_id = c(rep(0,floor(n_patients/2)), rep(1,n_patients - floor(n_patients/2)))

# Test for significant cpGs, corrected for number of ICRs
df_res <- tdhia::analyze_case_control(cpg_beta$cpg_beta_df, groud_id, n_adjust = nrow(icr_beta$icr_beta_df),
sum(df_res$adj_sig)

# Test for significant ICRs
df_res <- tdhia::analyze_case_control(icr_beta$icr_beta_df, groud_id)
sum(df_res$adj_sig)
```

Now we can simulate a associate study, where we check if there is a correlation between cpg methylation state and other patient variables.

First, a data frame is created with patient information, including any response variables or predictors. For

actual analysis, this data is usually imported from a csv file or similiar format.

```
df_study = data.frame(id = 1:n_patients)
df_study$race <- as.factor(sample(c(0,1,2), size = n_patients, replace = TRUE))
df_study$exposure<- runif(n = n_patients, min = 0, max= 10)
df_study$smoked <- as.factor(sample(c(0,1), size = n_patients, replace = TRUE))
df_study$Bead = factor(sample(x = c("83279", "83249", "72794", "5449"), size = n_patients, replace = T))
df_study$Col = factor(sample(x = 1:3, size = n_patients, replace = T))
df_study$Row = factor(sample(x = 1:6, size = n_patients, replace = T))
df_study$Patient_ID <- colnames(cpg_beta$cpg_beta_df)
df_study

rownames(df_study) <- colnames(cpg_beta$cpg_beta_df)

n.cores <- max(c(parallel::detectCores()-2,1))

df_complete <- tdhia::analyze_association(R = select(df_study, exposure),
                                          P = as.data.frame(t(cpg_beta$cpg_beta_df)),
                                          Pe = select(df_study, -exposure),
                                          family = "gaussian", n_p_adj = nrow(icr_beta$icr_beta_df), db_flag =
                                          rm.na.all = FALSE, verbose = TRUE, impute_na = TRUE,
                                          max_p_val = 0.05, n.cores = n.cores)
```

ICR-level regression tests

TDHIA also includes regression tools for testing ICR-level methylation patterns. Two direct ICR-level options are principal component regression and SKAT. A third option first tests individual CpGs with LIMMA and then combines CpG-level p-values within each ICR using Lancaster's method.

For binary traits, make sure the phenotype is coded as a two-level factor and set the reference group before modeling. The examples below use the simulated `smoked` variable as a placeholder binary outcome. For continuous outcomes, use `family = "continuous"` in `pc_regression_test()` and `out_type = "C"` in `skat_icr_test()`.

```
# Prepare analysis metadata and keep complete cases for the selected model.
icr_study <- df_study %>%
  dplyr::select(Patient_ID, smoked, exposure, race) %>%
  tidyr::drop_na()

rownames(icr_study) <- icr_study$Patient_ID
icr_study$smoked <- stats::relevel(as.factor(icr_study$smoked), ref = "0")

# Keep the same samples in the methylation matrix and study metadata.
shared_ids <- intersect(colnames(cpg_beta$cpg_beta_df), rownames(icr_study))
cpg_beta_icr <- cpg_beta$cpg_beta_df[, shared_ids, drop = FALSE]
icr_study <- icr_study[shared_ids, , drop = FALSE]
```

Principal component regression

Principal component regression summarizes the CpGs inside each ICR with enough principal components to explain the requested fraction of methylation variance. It then compares a full model that includes those methylation PCs to a reduced model with only the covariates.

```
# pc_regression_test() expects samples as rows and CpGs as columns.
cpg_beta_pc <- as.data.frame(t(cpg_beta_icr))
```

```

pc_icr_results <- tdhia::pc_regression_test(
  cpg_beta = cpg_beta_pc,
  m_value_transform = TRUE,
  data_norm_type = "n1",
  pct_variance = 0.80,
  df_study = icr_study,
  outcome = "smoked",
  covariates = c("exposure", "race"),
  Patient_ID = "Patient_ID",
  family = "binomial",
  icr_ids = NULL,
  min_cpg = 3,
  verbose = TRUE,
  n.cores = 1
)

pc_icr_results <- pc_icr_results %>%
  dplyr::left_join(
    tdhia::add_metadata_to_imp_sites(pc_icr_results$ICR_id, imp_type = "icr", db_flag = FALSE),
    by = c("ICR_id" = "icr_id")
  )

head(pc_icr_results)

```

SKAT ICR regression

SKAT tests each ICR as a set of CpG methylation values. It is useful when the CpGs inside an ICR may have heterogeneous effects.

```

skat_icr_results <- tdhia::skat_icr_test(
  cpg_betas = as.data.frame(cpg_beta_icr),
  df_study = icr_study,
  response = "smoked",
  predictors = c("exposure", "race"),
  method = "optimal.adj",
  out_type = "D",
  icr_ids = NULL,
  min_cpg = 3,
  db_flag = FALSE,
  m_value_transform = TRUE,
  scaling = TRUE,
  verbose = TRUE,
  n.cores = 1
)

skat_icr_results <- skat_icr_results %>%
  dplyr::left_join(
    tdhia::add_metadata_to_imp_sites(skat_icr_results$icr_id, imp_type = "icr", db_flag = FALSE),
    by = "icr_id"
  )

head(skat_icr_results)

```

LIMMA followed by Lancaster aggregation

Next we can perform a CpG-level DML analysis and ICR-level DMR analysis using a LIMMA-based workflow.

The CPG level analyses uses LIMMA for increasing power for differential methylation testing. This example queries external annotation services through downstream annotation helpers, so it is shown but not evaluated during package checks. Run it interactively when network access and annotation resources are available.

```
# Differential CpG analysis with LIMMA.
out_cpg <- tdhia::cpg_dml_test(
  df_study = df_study,
  predictors = c("smoked", "exposure", "race"),
  sample_name = "Patient_ID",
  cpg_beta = cpg_beta$cpg_beta_df,
  beadchip_correction = TRUE,
  db_flag = FALSE
)

out_cpg$plots$qq_p_value
out_cpg$plots$volcano
out_cpg$plots$manhattan
```

The ICR level analyses uses Lancaster p-value aggregation procedure to test for altered methylation state in ICRs.

```
# Differential icr analysis
out_icr <- tdhia::icr_dmr_test(df_dml = out_cpg$df_dml, chr_lens = out_cpg$chr_lens)

out_icr$plots$manhplot_icr
```